

Neural Networks, Turing Machines, and Exponential State Graphs

Corrected matrix-based analysis

31 July 2026

Abstract

The universal claim that every complex neural network must require exponentially many instructions when translated into a bounded linear-size Turing machine is false. A finite-precision neural network can be simulated by a deterministic register machine or Turing machine with polynomially many arithmetic operations. Exponential growth appears under a different requirement: explicitly enumerating a finite-state graph that preserves all distinguishable histories. Probabilistic execution is required only when the source model samples, or when abstraction deliberately discards information and represents the discarded state distribution by a Bayesian kernel.

1 Formal model

Consider a network with input dimension d_0 , layer widths $d_1, \dots, d_L$, and q -bit rational parameters:

$$h_0 = x, \quad z_l = W_l h_{l-1} + b_l, \quad h_l = \sigma_l(z_l), \quad y = R h_L.$$

All arithmetic is rounded to q bits, and every activation is computable in time polynomial in q . This includes fixed-point ReLU networks and quantized transformer blocks with a declared rounding rule.

2 Polynomial Turing-machine simulation theorem

Theorem. Let N be a q -bit finite-precision network with parameter count P and layer dimensions $d_0, \dots, d_L$. There exists a deterministic multi-tape Turing machine T_N computing exactly the rounded output of N in a number of steps polynomial in P , q , and input length. **Proof sketch.** Store each activation coordinate as a q -bit signed integer. To compute one matrix entry, read a weight and activation, multiply, add to an accumulator, and round. Schoolbook multiplication costs $O(q^2)$ bit operations. The number of scalar products is

$$M = \sum_{l=1}^L d_l d_{l-1}.$$

Thus affine layers cost $O(Mq^2)$ up to polynomial tape-management factors. Activations cost d_l times a polynomial in q , and the readout has the same form. Since M is at most the number of stored matrix entries P , total work is polynomial in P and q . Every operation is deterministic. QED.

3 Why the exponential instruction claim fails

A Turing machine has reusable tape and registers. It does not need one instruction or one control state for every possible activation vector. Matrix multiplication is a loop over shared instructions. Therefore a network with P parameters can have a program of size $O(P)$ plus a generic arithmetic interpreter, even if its activation space has size 2^{qd_L} .

Large configuration space $\neq$ large instruction set.

An explicit finite graph whose states are complete neural configurations can still be exponentially large. That is a lower bound on explicit-state representation, not on Turing-machine instruction count.

4 The stronger human-interpretability target

The clarified target is stricter than ordinary simulation. It requires a list or graph of readable opcodes with meaningful names, such as `READ_SUBJECT`, `COPY_AGREEMENT`, or `EMIT_ARE`, and it disallows hiding the neural arithmetic inside one opaque interpreter call.

For this target, define a name assignment N and a finite semantic graph G . Exactness means that every input history has the same output law in G as in the source model. If G is required to enumerate every distinguishable history, the identity-copy example below gives an exponential lower bound. If G merges histories, its transition must carry a conditional distribution over the hidden concrete states. That is the Bayesian branch of the tradeoff:

$$\text{explicit exact named states} \quad \text{or} \quad \text{named abstractions} + \text{conditional measures}.$$

This is a conditional theorem about explicit-state, human-readable representations, not a theorem about all Turing-machine programs. A readable algebraic macro such as `COPY_VECTOR(m)` is a third option; it remains compact because the macro denotes a family of operations rather than enumerating states.

5 An explicit-state exponential lower bound

Let the input be an m -bit string x . A small linear circuit copies it: $y = I_m x$, with $O(m)$ wires and operations. Require instead a finite-state transducer to read x once and emit its m bits later, one bit at a time, without a random-access tape.

After reading all bits, every pair $x \neq x'$ must reach different states because their required future output sequences differ. Hence

$$|\text{States}| \geq 2^m, \quad \text{while circuit description size} = O(m).$$

This proves exponential growth for a restricted explicit finite-state target. It does not prove exponential Turing-machine instructions: a Turing machine can store x on its tape and scan it during output.

6 Deterministic versus probabilistic targets

A deterministic rounded network has

$$s \xrightarrow{u} T(s, u), \quad y = E(s).$$

No Bayesian machine is needed for semantic equivalence. A probabilistic Turing machine is appropriate when the model samples:

$$P(y \mid s) = \text{softmax}(\text{logits}(s))_y.$$

After abstraction, probability can arise from lost information. If many concrete states map to abstract state a , then

$$K(a' \mid a, u) = \int \mathbf{1}[\alpha(T(s, u)) = a'] \rho_a(ds).$$

This probability is induced by abstraction; it does not imply that the original deterministic computation was probabilistic.

7 FLAN-T5 and Futamura specialization

An exact finite-precision FLAN-T5 simulator can be a deterministic register machine containing weights, encoder states, decoder KV cache, and arithmetic subroutines. Sampling its decoder produces a probabilistic Turing machine. PRSL-STACK-1 is a bounded specialization: it freezes a prompt domain, prefix horizon, and top- k quantized output distributions. Its current certificate covers eight prompts, top-2 branch expansion, horizon 3, and 56 states. At 2,850 bytes, maximum measured first-token total-variation error is 0.3650418194413511.

The first Futamura projection specializes an interpreter I with a frozen program P :

$$I(P, x) \longrightarrow P_{\text{compiled}}(x).$$

The second specializes a compiler with respect to the interpreter; the third specializes that compiler generator with respect to itself. These remove interpretation overhead, but do not evade information limits on the chosen finite graph or approximation domain.

8 Corrected theorem

A finite-precision neural network has a polynomial-size deterministic Turing-machine simulator in the number of stored parameters and bit precision. An explicit finite-state graph may require exponentially many states in input/history length. If the source uses sampling, or abstraction discards state information, the target semantics is naturally probabilistic.

The quantities that must not be conflated are program/instruction size, workspace/configuration count, and explicit graph-state count.

9 Limits and reproducibility

The proof assumes finite precision, computable activations, and a declared input representation. Exact real-valued networks are not finite binary objects until an encoding and arithmetic semantics are chosen. Lower bounds depend on the target model and available memory. Hence no unconditional exponential instruction lower bound follows merely from neural-network complexity.

The FLAN artifacts are the local `google/flan-t5-small` checkpoint and tokenizer. The bounded compiler and checker are `flan_stack.compile.py` and `check.flan_stack.py`; the Futamura test is `futamura_prsl.py`.